**Università degli Studi di Padova**
Laurea: Informatica
Corso: Ingegneria del Software
Anno Accademico: 2025/2026

**Gruppo: NullPointers Group**
Email: nullpointersg@gmail.com

# Specifica Tecnica

| | |
|---|---|
| Stato | In Approvazione |
| Versione | 0.1.0 |
| Data ultima modifica | 11/03/2026 |
| Redattori | Luca Marcuzzo |
| Verificatori | L. Pieripolli |
| Destinatari | NullPointers Group<br>Ergon Informatica Srl<br>Prof. Tullio Vardanega<br>Prof. Riccardo Cardin |

# Registro delle modifiche

| Vers | Data | Autore | Verificatore | Descrizione |
|---|---|---|---|---|
| 0.1.0 | 11-03 | L. Marcuzzo | L. Pieripolli | Creazione e stesura documento.<br>Redazione sezione 1 "Introduzione" e inizio sezione 2 "Tecnologie" |

Table 1: Registro delle modifiche

# Indice

**1 Introduzione** . . . . . **5**
1.1 Scopo del documento . . . . . 5
1.2 Scopo del progetto . . . . . 5
1.3 Glossario . . . . . 5
1.4 Riferimenti . . . . . 6
1.4.1 Riferimenti normativi . . . . . 6
1.4.2 Riferimenti informativi . . . . . 6

**2 Tecnologie** . . . . . **8**
2.1 Linguaggi di programmazione . . . . . 8
2.1.1 Python . . . . . 8
2.1.2 TypeScript . . . . . 8
2.2 Frontend . . . . . 8
2.2.1 React . . . . . 8
2.2.2 TailwindCSS . . . . . 9
2.2.3 Vite . . . . . 9
2.3 Backend . . . . . 9
2.3.1 FastAPI . . . . . 9
2.3.2 LangChain . . . . . 10
2.3.3 FAISS . . . . . 10
2.4 Database . . . . . 10
2.4.1 PostgreSQL . . . . . 10
2.5 Modelli di linguaggio . . . . . 11
2.5.1 GPT-5 . . . . . 11
2.6 Tecnologie per il deployment . . . . . 11
2.6.1 Docker . . . . . 11
2.7 Tecnologie di analisi . . . . . 11
2.7.1 Tecnologie di analisi statica . . . . . 11
2.7.1.1 SonarQube . . . . . 11
2.7.1.2 MyPy . . . . . 12
2.7.2 Tecnologie di analisi dinamica . . . . . 12
2.7.2.1 Pytest . . . . . 12
2.7.2.2 Vitest . . . . . 12

**3 Architettura Logica** . . . . . **13**

**4 Architettura di Sistema** . . . . . **14**

**5 Architettura di Deployment** . . . . . **15**

**6 Stato dei requisiti funzionali** . . . . . **16**

# Elenco delle Tabelle

1 Registro delle modifiche . . . . . 2

# 1 Introduzione

## 1.1 Scopo del documento

Il presente documento ha l'obiettivo di definire e descrivere l'architettura adottata per lo sviluppo$^G$ del sistema SmartOrder.
Il documento delinea le scelte architetturali, tecnologiche e di design adottate dal gruppo per la realizzazione del prodotto.
In particolare il documento si propone di definire:

- L'architettura del sistema, comprensiva delle componenti principali e delle loro interazioni;
- Le tecnologie e i framework$^G$ selezionati per lo sviluppo$^G$, con particolare attenzione alle motivazioni tecniche che ne hanno guidato l'adozione;
- I design pattern applicati e le ragioni tecnologiche che ne hanno guidato l'adozione;
- Il tracciamento dei requisiti, che attesta la copertura di quanto stabilito nel documento di Analisi dei Requisiti.

Il documento è soggetto ad aggiornamenti durante tutto il ciclo di vita del progetto$^G$, al fine di recepire nuove decisioni architetturali o eventuali modifiche legate all'evoluzione del progetto.

## 1.2 Scopo del progetto

SmartOrder è un prodotto ideato dall'azienda Ergon Informatica per automatizzare il processo$^G$ di gestione degli ordini di acquisto aziendali tramite l'analisi di input multimodali quali testo, immagini o audio. Attraverso l'utilizzo del Natural Language Processing$^G$ (NLP$^G$), SmartOrder interpreta queste richieste spesso non strutturate, estraendo le informazioni rilevanti quali prodotti e quantità, trasformandole in ordini strutturati pronti per l'integrazione nei sistemi ERP$^G$ aziendali.

## 1.3 Glossario

Per assicurare una comprensione univoca dei termini usati in questo Piano di Qualifica$^G$, si fornisce un glossario dedicato.
La nomenclatura utilizzata per segnalare che la definizione di una parola è contenuta nel glossario è la seguente:

termine$^G$

I termini sono ordinati alfabeticamente per facilitarne la consultazione e vengono inclusi sia termini tecnici che acronimi significativi.
Il gruppo si impegna a visionare il Glossario periodicamente, per permettere la più completa comprensione di ogni tipo di documento pubblicato e per mantenere un allineamento semantico costante tra tutti i partecipanti al progetto.

## 1.4 Riferimenti

### 1.4.1 Riferimenti normativi

- **Norme di Progetto$^G$, versione 1.1.0**
  https://nullpointersgroup.github.io/Documentazione/output/PB/Documenti%20Interni/Norme_di_Progetto.pdf
  **Ultima consultazione: 12 gennaio 2026**

- **Capitolato$^G$ C8 - Ergon Informatica Srl - SmartOrder**
  https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C8.pdf
  **Ultima consultazione: 3 dicembre 2025**

### 1.4.2 Riferimenti informativi

- **Glossario, versione 1.0.0**
  https://nullpointersgroup.github.io/Documentazione/output/PB/Documenti%20Interni/Glossario.pdf
  **Ultima consultazione: 11 marzo 2026**

- **Analisi dei requisiti$^G$, versione 1.1.2**
  https://nullpointersgroup.github.io/Documentazione/output/PB/Documenti%20Interni/Analisi_dei_Requisiti.pdf
  **Ultima consultazione: 11 marzo 2026**

- **Analisi dei requisiti$^G$, SWE 2025-2026**
  https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T05.pdf
  **Ultima consultazione: 11 marzo 2026**

- **Software Architecture Patterns, SWE 2025-2026**
  https://www.math.unipd.it/~rcardin/swea/2022/Software%20Architecture%20Patterns.pdf
  **Ultima consultazione: 11 marzo 2026**

- **Dependency Injection, SWE 2025-2026**
  https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Architetturali%20-%20Dependency%20Injection.pdf
  **Ultima consultazione: 11 marzo 2026**

- **Design Pattern Creazionali, SWE 2025-2026**
  https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%20Creazionali.pdf
  **Ultima consultazione: 11 marzo 2026**

- **Design Pattern Strutturali, SWE 2025-2026**
  https://www.math.unipd.it/~rcardin/swea/2022/Design%20Pattern%

20Strutturali.pdf

**Ultima consultazione: 11 marzo 2026**

# 2 Tecnologie

In questa sezione vengono descritte le tecnologie adottate per lo sviluppo$^G$ del progetto$^G$ **SmartOrder**, suddivise per ambito di utilizzo. Per ciascuna tecnologia vengono riportate la versione di riferimento, la documentazione ufficiale e il ruolo ricoperto all'interno del sistema.

## 2.1 Linguaggi di programmazione

### 2.1.1 Python

- **Versione:** 3.14.3
- **Documentazione:** https://docs.python.org/3/

Python$^G$ è un linguaggio di programmazione ad alto livello, interpretato e multiparadigma, progettato per favorire la leggibilità del codice e la rapidità nello sviluppo$^G$ delle applicazioni.
Nel progetto$^G$ **SmartOrder** viene utilizzato principalmente per lo sviluppo$^G$ del backend$^G$ applicativo e per l'integrazione con i modelli di intelligenza artificiale. In particolare, gestisce la logica di elaborazione degli ordini, l'implementazione delle API$^G$ tramite FastAPI e l'interazione con il database$^G$ e con i sistemi di memorizzazione vettoriale. Grazie al suo ampio ecosistema di librerie, Python$^G$ risulta particolarmente adatto allo sviluppo$^G$ di applicazioni basate su tecniche di machine learning$^G$ e intelligenza artificiale.

### 2.1.2 TypeScript

- **Versione:** 5.9
- **Documentazione:** https://www.typescriptlang.org/docs/

TypeScript è un linguaggio di programmazione open source che estende JavaScript introducendo la tipizzazione statica, interfacce, tipi complessi e generics. I controlli sui tipi a compile-time riducono gli errori comuni legati all'assenza di tipizzazione e migliorano la leggibilità, la manutenibilità e la robustezza del codice.
Nel progetto$^G$ viene utilizzato per gestire dati strutturati complessi come ordini, prodotti, quantità e validazioni, contribuendo a mantenere coerenza tra frontend$^G$ e backend$^G$ e a ridurre il rischio di errori nella comunicazione tramite API.

## 2.2 Frontend

### 2.2.1 React

- **Versione:** 19.2

- **Documentazione:** https://it.react.dev/reference/react

React$^G$ è una libreria JavaScript open source per la costruzione di interfacce utente basate su un modello dichiarativo e su componenti indipendenti e riutilizzabili. Utilizza un DOM virtuale per aggiornare selettivamente solo le parti dell'interfaccia effettivamente modificate, ottimizzando le prestazioni di rendering.
Non essendo opinionata nella scelta delle librerie di routing o di gestione dello stato, non impone vincoli architetturali, permettendo di strutturare il frontend$^G$ in modo modulare e adattarlo alle esigenze dell'applicazione.
**Librerie utilizzate**
*inserire le librerie utilizzate*

### 2.2.2 TailwindCSS

- **Versione:** 4.2
- **Documentazione:** https://tailwindcss.com/docs/installation/using-vite

TailwindCSS è un framework$^G$ CSS utility-first che mette a disposizione un ampio insieme di classi di utilità che applicano singole proprietà CSS direttamente nel markup. Questo approccio consente di costruire interfacce in modo rapido e coerente, evitando la scrittura di fogli di stile personalizzati e complessi, e favorendo una maggiore manutenibilità del codice di presentazione.

### 2.2.3 Vite

- **Versione:** 7.3.1
- **Documentazione:** https://vite.dev/guide/#scaffolding-your-first-vite-project

Vite è uno strumento di build$^G$ per lo sviluppo$^G$ di applicazioni web moderne. Sfrutta i moduli ES nativi del browser per avviare l'ambiente di sviluppo$^G$ con tempi di avvio ridotti, offrendo hot module replacement per aggiornamenti immediati durante lo sviluppo. Supporta la configurazione tramite plugin, l'ottimizzazione automatica delle dipendenze e la generazione di bundle di codice ottimizzati tramite Rollup.

## 2.3 Backend

### 2.3.1 FastAPI

- **Versione:**
- **Documentazione:** https://fastapi.tiangolo.com

FastAPI è un framework$^G$ web per Python$^G$ utilizzato per sviluppare API$^G$ REST$^G$ ad alte prestazioni. Si basa sullo standard ASGI e utilizza librerie come Pydantic e Starlette per la gestione delle richieste e la validazione$^G$ dei dati.

Nel progetto$^G$ **SmartOrder** viene utilizzato per implementare il backend$^G$ dell'applicazione e gestire la comunicazione tra frontend$^G$, moduli di elaborazione e database. Inoltre, consente di integrare facilmente i modelli di intelligenza artificiale$^G$ e di generare automaticamente la documentazione delle API$^G$ tramite strumenti basati su OpenAPI.

### 2.3.2 LangChain

- **Versione:**
- **Documentazione:** https://python.langchain.com

LangChain è un framework$^G$ open source progettato per facilitare lo sviluppo$^G$ di applicazioni basate su Large Language Models (LLM$^G$). Fornisce strumenti per gestire prompt, orchestrare le chiamate ai modelli e integrare i modelli di linguaggio con fonti di dati esterne.
Nel progetto$^G$ **SmartOrder** viene utilizzato per gestire l'interazione con il modello di linguaggio e costruire le pipeline di elaborazione necessarie per interpretare gli input degli utenti ed estrarre le informazioni rilevanti dagli ordini, trasformandole in dati strutturati utilizzabili dal sistema.

### 2.3.3 FAISS

- **Versione:**
- **Documentazione:** https://faiss.ai/index.html

FAISS (Facebook AI Similarity Search) è una libreria open source sviluppata da Meta per l'indicizzazione e la ricerca efficiente di vettori ad alta dimensionalità, utilizzata principalmente per operazioni di similarity search tra embedding.
Nel progetto$^G$ **SmartOrder** viene utilizzato come database$^G$ vettoriale per memorizzare e confrontare gli embedding generati dai modelli di linguaggio. Questo consente di individuare prodotti simili a partire da descrizioni testuali e migliorare l'accuratezza del matching tra le richieste dell'utente e il catalogo aziendale.

## 2.4 Database

### 2.4.1 PostgreSQL

- **Versione:**
- **Documentazione:** https://www.postgresql.org/docs/

PostgreSQL$^G$ è un sistema di gestione di basi di dati relazionali open source conforme agli standard SQL$^G$, noto per la sua affidabilità e robustezza nella gestione dei dati.

Nel progetto$^G$ **SmartOrder** viene utilizzato come database$^G$ principale per la memorizzazione persistente delle informazioni dell'applicazione, tra cui il catalogo dei prodotti, gli ordini generati dal sistema e i dati associati agli input elaborati. L'utilizzo di PostgreSQL$^G$ consente di garantire integrità, consistenza e affidabilità nella gestione dei dati e si integra facilmente con il backend$^G$ sviluppato in Python.

## 2.5 Modelli di linguaggio

### 2.5.1 GPT-5

- **Versione:**
- **Documentazione:** https://platform.openai.com/docs

GPT-5 è un Large Language Model basato sull'architettura Transformer, progettato per comprendere e generare testo in linguaggio naturale. Grazie all'addestramento su grandi quantità di dati testuali, è in grado di interpretare il significato delle richieste degli utenti e di estrarre informazioni rilevanti dal testo.
Nel progetto$^G$ **SmartOrder** viene utilizzato per analizzare gli input degli utenti espressi in linguaggio naturale e trasformarli in dati strutturati utilizzabili dal sistema, individuando informazioni come prodotti, quantità e altri dettagli necessari alla generazione degli ordini.

## 2.6 Tecnologie per il deployment

### 2.6.1 Docker

- **Versione:**
- **Documentazione:** https://docs.docker.com

Docker$^G$ è una piattaforma open source che consente di sviluppare ed eseguire applicazioni all'interno di container, ambienti isolati che includono tutte le dipendenze necessarie al funzionamento del software.
Nel progetto$^G$ **SmartOrder** viene utilizzato per containerizzare i diversi componenti dell'applicazione, come backend$^G$, frontend$^G$ e database$^G$, garantendo un ambiente di esecuzione coerente tra sviluppo$^G$, testing e deployment e semplificando il processo$^G$ di distribuzione$^G$ del sistema.

## 2.7 Tecnologie di analisi

### 2.7.1 Tecnologie di analisi statica

#### 2.7.1.1 SonarQube

- **Versione:**
- **Documentazione:** https://docs.sonarsource.com/sonarqube

SonarQube è una piattaforma di analisi statica del codice che consente di valutare la qualità del software individuando potenziali errori, vulnerabilità di sicurezza e problemi di manutenibilità.
Nel progetto$^G$ **SmartOrder** viene utilizzato per monitorare automaticamente la qualità del codice prodotto durante lo sviluppo$^G$, permettendo di individuare precocemente eventuali criticità e mantenere elevati standard qualitativi nel progetto.

#### 2.7.1.2 MyPy

- **Versione:**
- **Documentazione:** https://mypy.readthedocs.io

MyPy è uno strumento di analisi statica per Python$^G$ che verifica$^G$ la correttezza delle annotazioni di tipo presenti nel codice sorgente.
Nel progetto$^G$ **SmartOrder** viene utilizzato per controllare la coerenza dei tipi nel backend$^G$ sviluppato in Python$^G$, contribuendo a individuare errori prima dell'esecuzione del programma e migliorando la robustezza del codice.

### 2.7.2 Tecnologie di analisi dinamica

#### 2.7.2.1 Pytest

- **Versione:**
- **Documentazione:** https://docs.pytest.org

Pytest è un framework$^G$ di testing per Python$^G$ che consente di scrivere ed eseguire test$^G$ automatici per verificare il corretto funzionamento del codice.
Nel progetto$^G$ **SmartOrder** viene utilizzato per testare le componenti backend$^G$ sviluppate in Python$^G$, verificando il comportamento delle API$^G$ e delle funzioni di elaborazione per garantire l'affidabilità del sistema.

#### 2.7.2.2 Vitest

- **Versione:**
- **Documentazione:** https://vitest.dev

Vitest è un framework$^G$ di testing per applicazioni JavaScript e TypeScript integrato con l'ecosistema Vite, progettato per eseguire test$^G$ in modo rapido ed efficiente.
Nel progetto$^G$ **SmartOrder** viene utilizzato per testare il codice frontend$^G$ sviluppato con React$^G$ e TypeScript, verificando il corretto funzionamento delle componenti dell'interfaccia utente e delle logiche di interazione con il backend.

# 3 Architettura Logica

# 4 Architettura di Sistema

# 5 Architettura di Deployment

# 6 Stato dei requisiti funzionali